



OSTBAYERISCHE  
TECHNISCHE HOCHSCHULE  
REGENSBURG

ELEKTRO- UND  
INFORMATIONSTECHNIK

# **PRAKTIKUM INFORMATIK 2**

M. Niemetz

## **Übung: Verkettete Listen und Rekursion**

Klausurdatenverwaltung

Juni 2026, Version: 1.6

Fakultät Elektro- und Informationstechnik

## Vorbereitung

- ☐ ☕ **Schutz gegen Mehrfachinklusion:** Lesen Sie ggf. nach, wozu ein Schutz gegen Mehrfachinklusion in Headerdateien benötigt wird, wie er zu implementieren ist und machen Sie sich klar, wie er funktioniert. . . . . 3
- ☐ ☕ **ifstream-Klasse:** Schlagen Sie die Eigenschaften der Klasse `ifstream` nach, die den Inhalt einer Datei als Eingabestream zur Verfügung stellt. Insbesondere sollten Sie den **operator>>** und die Methode `eof()` rechechieren. . . . . 5
- ☐ ☕ **Modifizierer für ostream-Objekte:** Schlagen Sie die Verwendung der Modifizierer `setw`, `setprecision` und `fixed` für `ostream`-Objekte nach. . . . . 6

## 1 Einleitung

In dieser Aufgabe sollen verkettete Listen, Dateioperationen (Einlesen) sowie rekursive Algorithmen in einem objektorientierten Programmierumfeld mit untereinander wechselwirkenden Objekten verschiedener Klassen geübt werden.

Als Übungsbeispiel kommt dabei eine Verwaltung für Klausurergebnisse zum Einsatz. Ziel dieser Anwendung ist es, aus einer Datei die Klausurergebnisse in eine verkettete Liste einzulesen und dann alphabetisch zu sortieren, damit die Eintragung der Noten ermöglicht wird.

Die Abbildung 1 zeigt exemplarisch das Format der Daten, die eingelesen werden sollen. Die Daten sind zeilenweise angeordnet, wobei jeweils zunächst der Name des Prüflings, dann die Punktezahl und dann die Note, multipliziert mit 10 (zur Vermeidung des Kommas), angegeben sind.

**Hinweis:** Um den Einlesevorgang zu erleichtern, enthalten die Namen keine Leerzeichen sondern ggf. Unterstriche zur Trennung von Namensbestandteilen.

Da die Anzahl der Teilnehmer völlig offen ist, sollen die Daten beim Einlesen in einer verketteten Liste gespeichert werden.

```
Zumbalu_Zahira 70 13
Beispiel_Peter 43 13
Mirakel_Paula 60 10
Sonstwer_Petra 55 13
Sankt_Martin 30 23
```

Abbildung 1: Beispiel des Dateiinhalts der verarbeitet werden soll.

Für die Listenelemente kommt hierbei ein Klassentyp zum Einsatz, so dass es möglich ist, die Listenelemente mit den benötigten Methoden auszustatten.

Die Abbildung 2 visualisiert die entsprechende Realisierung der Implementierung in einem UML-Klassendiagramm.

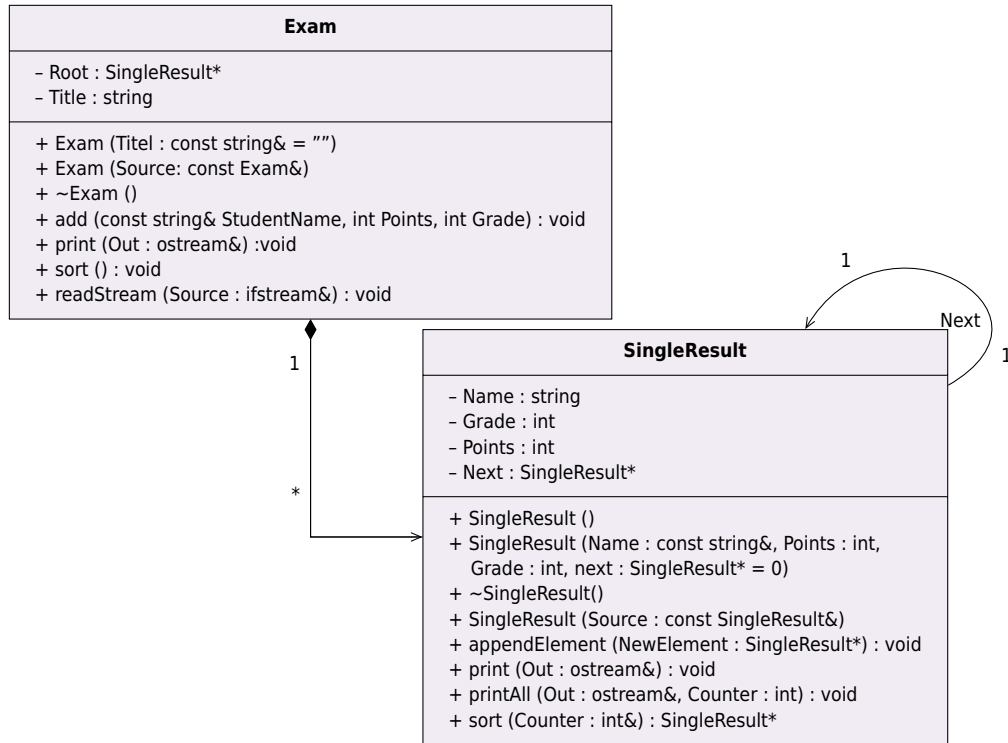


Abbildung 2: Darstellung der Architektur der Anwendung zur Verwaltung der Klausurergebnisse als UML Klassendiagramm.

## 2 Vorbereiten des Projekts

Legen Sie ein neues leeres Codeblocks-Projekt in Ihrem Git-Projektverzeichnis auf Ihrem Arbeits-PC an.

Legen Sie dann in Ihrem Projektverzeichnis eine Datei mit Testdaten, wie in Abbildung 1 gezeigt, an und fügen Sie alle Dateien zu Ihrem Projekt hinzu.

Stellen Sie sicher, dass nicht nur die Dateien, sondern auch die .cbp-Projektdatei korrekt gespeichert sind (zum Beispiel indem Sie das Projekt in CodeBlocks schließen).



Bereiten Sie das Projekt für die Entwicklung vor.



Erzeugen Sie eine neue Version und laden Sie sie auf den git-Server hoch.

### 3 Deklaration der Klassen

Definieren Sie nun die Klassen. Legen Sie für jede Klasse ein eigenes Paar an .h und .cpp Dateien an.

**Hinweis:** Achten Sie bei der Erstellung der Headerdateien auf eine korrekte Implementierung des Schutzes gegen Mehrfachincludes!

**Wichtig:** Überlegen Sie sich, welche Methoden **const** sein müssen, da diese Information nicht in das UML-Diagramm aufgenommen wurde!



Erzeugen Sie eine neue Version und laden Sie sie auf den Server hoch.

Definieren Sie die Klassen [Exam](#) und [SingleResult](#).

#### Recherche ☕

Schutz gegen Mehrfachinklusion

### 4 Die Implementierung

**Hinweis:** Diese Anleitung gibt nur grobe Hinweise – erarbeiten Sie sich die Lösung selbst! Insbesondere sollten Sie bei jedem Implementierungsschritt versuchen, Ihre Arbeit sofort zu testen.



Nach jedem erfolgreich getesteten Entwicklungsschritt sollten Sie eine neue Version erzeugen.

#### 4.1 Konstruktoren

Beginnen Sie mit der Implementierung der Konstruktoren der beiden Klassen.

Beim Copy-Konstruktor von [Exam](#) benötigen wir eine tiefe Kopie der verketteten Liste.

Um die Arbeit möglichst einfach zu gestalten, gehen Sie dabei **rekursiv** vor:

Der Copy-Konstruktor von [Exam](#) legt (bei Bedarf) lediglich das erste [SingleResult](#)-Objekt als Kopie der zu kopierenden Liste an. Verwenden Sie dabei unbedingt den Copy-Konstruktor der Klasse [SingleResult](#) und *keine* Zuweisung.

Danach erledigt Der Copy-Konstruktor seine sonstigen Aufgaben, ohne sich um den Rest der Liste zu kümmern.

Der Copy-Konstruktor von [SingleResult](#) wiederum kopiert (abgesehen von seinen übrigen Aufgaben) nur das dem aktuellen Element nachfolgende Objekt (sofern es existiert).



Implementieren Sie die benötigten Konstruktoren von [Exam](#) und [SingleResult](#).

Damit verursacht die Kopie im Copy-Konstruktor den Aufruf des Copy-Konstruktors im Folgeelement und so weiter: Die gesamte Liste wird rekursiv kopiert.

**Wichtig:** Achten Sie bei allen Konstruktoren darauf, immer und in jedem Fall alle Attribute des konstruierten Objekts zu initialisieren!

**Wichtig:** Verwenden Sie soweit technisch möglich die Initialisierersliste für die Implementierung des Konstruktors und verzichten Sie weitestgehend auf die Nutzung des Anweisungsblocks

**Hinweis:** Machen Sie sich klar, wie die Copy-Konstruktoren zusammenarbeiten, um eine Liste mit 4 Elementen tief zu kopieren.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Machen Sie sich das Zusammenwirken der Konstruktoren von [Exam](#) und [SingleResult](#) anhand einer Skizze klar.

## 4.2 Destruktoren

Gehen Sie zur Implementierung der Destruktoren beider Klassen [Exam](#) und [SingleResult](#) analog vor.

**Hinweis:** „Analog“ will in diesem Fall sagen, Sie sollen die selbe Idee auf das Problem der Destruktoren anwenden, die auch bei den Copy-Konstruktoren zum Einsatz kam.

Überlegen Sie sich, wie Sie den Destruktor testen können und führen Sie den Test durch.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Implementieren Sie die Destruktoren von [Exam](#) und [SingleResult](#).

## 4.3 Einfügen neuer Elemente

Fahren Sie nun als nächstes mit der Implementierung der Methode [SingleResult::appendElement](#) fort. Auch hier wählen Sie wieder einen rekursiven Ansatz:

Diese Methode soll dazu den übergebenen Pointer im Attribut [Next](#) speichern, sofern dieses Datenelement noch nicht mit einem gültigen Pointer belegt ist.

Ist das Attribut [Next](#) jedoch bereits mit einem Pointer auf ein [SingleResult](#)-Objekt belegt, wird der übergebene Pointer an die Methode [appendElement](#) dieses Objekts weitergereicht.

Nun können Sie in der Klasse [Exam](#) die notwendigen Methoden zum Einlesen der Daten implementieren.



Implementieren Sie die Methoden zum Einfügen neuer Elemente in die Liste. Verwenden Sie einen rekursiven Algorithmus.

Beginnen Sie mit `Exam::add`. Diese Methode soll mit den ihr übergebenen Daten ein neues `SingleResult`-Objekt anlegen und dieses entweder an die Methode `appendElement` des ersten Listenelements übergeben (sofern es existiert), oder das neue Element als erstes Listenelement speichern.

**Frage:** Überlegen Sie sich, wie dieser rekursive Ansatz funktioniert. Wo ist die für rekursive Algorithmen zwingend erforderliche Abbruchbedingung zu finden?

Abschließend ist noch der Einlesevorgang aus der Datei zu implementieren. Hierfür kommt ein `ifstream`-Objekt zum Einsatz, das der Methode `Exam::readStream` als Parameter übergeben wird und das Einlesen von Daten über den `operator>>` unterstützt.

Lesen Sie die jeweils zu einem Datensatz (Zeile) gehörigen Werte in Variablen ein und prüfen Sie, ob zwischenzeitlich das Dateiende erreicht wurde, indem Sie die Methode `eof` des `ifstream`-Objekts aufrufen.

Falls das Dateiende erreicht ist, verlassen Sie die Funktion. Andernfalls übergeben Sie die gelesenen Werte an `Exam::add` und fahren mit dem Einlesen fort.

Testen Sie Ihren Entwicklungsschritt.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

**Recherche** ☕  
ifstream-Klasse

## 4.4 Die Ausgaberroutinen

Schreiben Sie die Methode `SingleResult::print`, welche ein Einzelergebnis in der Form

Name: Zumbalu\_Zahira; Punkte: 70; Note: 1.3

ausgibt.

**Hinweis:** Um eventuelle Rundungsfehler zu vermeiden, wird die Note in einer Variable des Typs `int` gespeichert, indem die Note mit einem Faktor 10 multipliziert wird. Für die Ausgabe soll allerdings wieder die gewohnte Darstellung als Dezimalzahl mit Nachkommastellen realisiert werden.

**Hinweis:** Verwenden Sie dabei den `ostream`-Modifizierer `setw` der von der `ostream`-Klasse unterstützt wird, um die Feldbreite für Name, Punkte und Note auf jeweils 25, 3 und 5 Zeichen einzustellen. Verwenden Sie weiterhin die Modifizierer `setprecision` und `fixed` um die angegebene Darstellung der Note mit der richtigen Anzahl an Nachkommastellen zu erreichen.

Am Ende der Ausgabe fügen Sie einen Zeilenwechsel ein.



Implementieren Sie die Methoden zum Ausgeben der Liste. Verwenden Sie einen rekursiven Algorithmus.

**Recherche** ☕

Modifizierer für `ostream`-Objekte

Nun schreiben Sie die Methode `SingleResult::printAll`, welche zunächst die laufende Nummer des Eintrags (s. Parameterliste) ausgibt, dann `print` aufruft und schließlich (sofern nötig) `printAll` des nächsten Listeneintrags aufruft, dabei aber die laufende Nummer um eins erhöht.

Schließlich implementieren Sie die noch fehlende Methode `Exam::print`, indem Sie zunächst eine Überschrift, welche das Attribut `Title` enthält, gefolgt von einer Trennlinie ausgeben.

Dann rufen Sie (bei Bedarf) `printAll` des ersten Listenelements auf.

Zum Abschluss der Ausgabe fügen Sie noch eine Trennlinie an.

Das Ganze sollte dann am Ende so aussehen, wie es die Abbildung 3 anhand einer Beispieldatensammlung zeigt.

```

Klausurergebnisse der Prüfung Informatik 2
-----
Nr. 1 Name:           Zumbalu_Zahira; Punkte: 70; Note: 1.3
Nr. 2 Name:           Beispiel_Peter; Punkte: 43; Note: 1.3
Nr. 3 Name:           Mirakel_Paula; Punkte: 60; Note: 1.0
Nr. 4 Name:           Sonstwer_Petra; Punkte: 55; Note: 1.3
Nr. 5 Name:           Sankt_Martin; Punkte: 30; Note: 2.3
-----

```

Abbildung 3: Beispielausgabe

**Frage:** Überlegen Sie sich, wie die rekursive Ausgabe funktioniert. Wo ist die Abbruchbedingung? Wie funktioniert das mit der Ausgabe der laufenden Nummer?



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

## 4.5 Sortierung

Für die Sortierung kommt eine Variante von Bubblesort zum Einsatz, die hier am einfachsten ebenfalls **rekursiv** implementiert wird.

- Die Methode `SingleResult::sort` ruft zunächst (falls möglich) `sort` des Nachfolgeelements in der Liste auf (mit `Counter` als Referenzparameter). Sollte der Rückgabewert nicht der Nullpointer sein, wird der Rückgabewert als neuer Wert für das Attribut `Next` übernommen.
- Danach prüft die Methode, ob der eigene `Name` kleiner ist als der des nachfolgenden Objekts in der Liste (sofern verfügbar).

**Reihenfolge nicht passend:** Das Element und sein `Next` Nachbar sind zu vertauschen!

- Der `Counter` wird inkrementiert.



Implementieren Sie die Methoden zum Sortieren der Liste. Verwenden Sie einen rekursiven Algorithmus.

- Die beiden Elemente werden vertauscht, indem ihre beiden **Next**-Pointer geeignet gesetzt werden.
- Abschließend liefert die Funktion als Rückgabewert den Pointer auf den *bisherigen* **Next** Nachbarn an den Aufrufer zurück, um diesen über den neuen Nachbarn zu informieren.

**Reihenfolge passend:** Das Element und sein **Next** Nachbar sind nicht zu vertauschen: Die Funktion liefert einen Nullpointer zurück, um zu informieren, dass keine Vertauschung nötig war.

**Hinweis:** Diese Stelle ist etwas trickreich. Allerspätestens jetzt empfiehlt sich die Anfertigung einer Skizze!

Damit ist nun alles vorbereitet für die Implementierung von **Exam**: : **sort**. Diese Methode ruft (mit einer auf Null gesetzten Zählervariable als Parameter) **sort** des ersten Listenelements (sofern verfügbar) auf.

Falls der Rückgabewert nicht der Nullpointer ist, wird der Rückgabewert in **Root** gespeichert.

Der gesamte Sortiervorgang wird so lange wiederholt, bis der jeweils zu Beginn des Suchschritts auf Null gesetzte Zähler nicht mehr hochgezählt wird und bis zum Ende des Sortierschritts Null bleibt.

Dann ist die Sortierung vollständig abgeschlossen.

**Hinweis:** Machen Sie sich klar, wie diese Version von Bubblesort funktioniert!



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

## 4.6 Operatoren

Fügen Sie nun Ausgabeoperatoren << für die beiden Klassen hinzu und verwenden Sie sie in Ihrem Programm.

Achten Sie darauf, dass die Verkettung von Operatoren wie bei **a=b+c+d+e** korrekt funktioniert.

Statten Sie beide Klassen mit einem Ausgabeoperator aus und verwenden Sie sie.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.